CrumbTrails - A Route Tracking Application

Group Members:
Adam James Roy, Ayra Baig, Ibrahim Sajid

July 10, 2026

Abstract

CrumbTrails is an Android application that aims to solve one problem for runners, drivers, motorcyclists, and many more: tracking your aimless routes. Whether it is a joyride in a car or trying to find a more interesting running route, CrumbTrails will remember the trail that you have traveled, allowing users to look back on their route and place photos along that path automatically. Once having saved a route, these are also able to be shared amongst other users in the CrumbTrail community, finding more beautiful scenery, tasty pizza joints... Only your imagination is the limit!

Contents

1 Project Overview

1.1 Main Features and Functionality

1.2 Motivation for Creation

2 Technical Architecture

2.1 Frontend

2.2 Backend

Important Data Classes

When saving each route to the local Room database, the data class below, **TrackedRoute**, was created:

```
data class TrackedRoute(
    val id: String,
    val tripName: String,
    val startedAtEpochMillis: Long,
    val endedAtEpochMillis: Long,
    val trackedRoute: List<LatLng>
)
```

The tracked data class has a couple unique traits in the form of its parameters. First, when the time stamp for beginning the trip and ending is measured by **Epoch time**. This is the time elapsed in milliseconds from January 1st, 1970. These two long values will be used to determine the time of the trip, and how long the trip lasted for. Secondly, the trackedRoute parameter is a list of tuples representing the gathered points latitude and longitude values (decimal form). Entering this into the database then requires encoding of the trackedRoute LatLng list, into a string using Google Maps PolyUtil API. *See Backend Implementation

3 API Design

3.1 Backend Implementaion

The following are functions that are substantial to for understanding the design choices taken for this project.

Function: `InitializeDB()`

A Function Description
Source: database/database.go, line 11

Parameters

Name	Type	Description
`No parameters`		

Returns *sql.DB - A pointer to a SQL database object wired up with the Render database url and the proper table entries for the menu

3.2 Frontend Implementation

Similar to the previous Backend Implementation section, the frontend

Function: `InsertEntry(item, code, cost, imagePath)`

A Function Description
Source: crudAPI/crudMenu/create.go, line 15

Parameters

Name	Type	Description
`item`	string	Menu item name
`code`	string	Unique item code
`cost`	cost	Cost of item in Kyat
`imagePath`	string	String for the path to image in Render

Returns

4 Requirements and Installation Instructions

4.1 System Requirements

4.2 Installation Instructions

5 Testing

5.1 Summary

The following is our testing method for ensuring a complete, safe application environment:

1.

5.2 Known Limitations

6 Attributions and Citations

6.1 Cited Works

As the project virtually entirely used Google API platforms, when the Android Developers Documentation lacked depth, Gemini was resorted to to answer lingering questions. When possible, learning from the documentation was most crucial to learning the ins and outs of Android Studio, Kotlin, and Jetpack Compose. The following were used in this development process:

1. **States With Jetpack Compose**: `https://developer.android.com/develop/ui/compose/state?hl=ja`
2. **Dialog Composables**: `https://developer.android.com/develop/ui/compose/components/dialog?hl=ja`
3. **Type Converters for Room**: `https://developer.android.com/training/data-storage/room/referencing-data?hl=ja`
4. **General Room Explanation (DAO, Entities, Repositories)**: `https://developer.android.com/training/data-storage/room/defining-data?hl=ja`
5. **Firebase - Application Connection**: `https://firebase.google.com/docs/auth/android/start?hl=ja`
6. **Embedded Photo Picker**: `https://developer.android.com/training/data-storage/shared/photo-picker/embedded?hl=ja`
7. **NavController and NavHost Setup**: `https://developer.android.com/codelabs/basic-android-kotlin-compo hl=ja`
8. **StateFlow and LiveData**: `https://developer.android.com/kotlin/flow/stateflow-and-sharedflow?hl=ja`
9. **Switching to Foreground Service**: `https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start`

6.2 Contributions

Adam Created the local Room database structure, their accompanying files, and focused on creating the layout of the API. Implemented the Composables necessary to get the general functionality including for tracking, history, feed, and authentication.

Ayra

Ibrahim ☺